DENA: A System-Level Architecture for Distributed Expert AI Systems

Preprint version 0.3 – figure-explained edition

Harsha Sai Reddy Eada
Independent Researcher
harshasaireddyind@gmail.com
github.com/harsha-sai-q/dena
Code and benchmark archive: 10.5281/zenodo.19841838

April 2026

Abstract

Modern artificial intelligence systems are increasingly built from multiple models, tools, retrievers, verifiers, and feedback mechanisms rather than from a single monolithic model alone. However, these systems are often implemented as task-specific pipelines without a shared formal structure for expert selection, expert metadata, aggregation, verification, memory, and feedback-driven updates. This paper proposes DENA (Distributed Expert Neural Architecture), a system-level architecture for modular AI systems composed of independently trained expert subsystems coordinated through task decomposition, dynamic routing, weighted aggregation, verification, memory, and feedback-driven improvement.

DENA does not introduce a new neural computation primitive such as attention, convolution, or recurrence. Instead, its contribution is the formalization and unification of patterns already emerging across expert systems, model orchestration frameworks, tool-using agents, retrieval-augmented generation pipelines, and Mixture-of-Experts-inspired systems. The paper defines the DENA architecture, presents a mathematical formulation for expert routing and aggregation, implements a minimal prototype, and introduces DENA-Bench v0, a controlled benchmark for measuring routing quality, output quality, estimated expert activation cost, and latency.

Preliminary experiments using simplified expert modules show that sparse expert routing can approximate all-expert execution in a controlled setting while reducing estimated expert activation cost. These results are not intended to demonstrate superiority over production dense models or large-scale Mixture-of-Experts systems. Rather, they demonstrate that DENA is implementable, measurable, interpretable, and suitable as a research framework for studying distributed expert AI systems.

Keywords: Distributed Expert Neural Architecture, modular AI, expert routing, model orchestration, Mixture of Experts, AI systems, verification, feedback learning, benchmarking.

Contents

1 Introduction **2**

1.1 Scope of the Claim 2

1.2 Contributions 2

2 Background and Related Work **3**
2.1 Dense Transformer Models . . . 3
2.2 Mixture of Experts . . . 3
2.3 Retrieval-Augmented Generation . . . 3
2.4 Tool-Using Agents . . . 4
2.5 Model Orchestration Systems . . . 4
2.6 Multi-Agent Systems . . . 4
2.7 Federated and Distributed Learning . . . 4
2.8 Knowledge Distillation and Human Feedback . . . 4

3 DENA Architecture **5**
3.1 High-Level Flow . . . 5
3.2 How to Interpret the Architecture Diagram . . . 5
3.3 Core Components . . . 6

4 Mathematical Formulation **6**
4.1 System Definition . . . 6
4.2 Preprocessing and Task Decomposition . . . 7
4.3 Expert Routing . . . 7
4.4 Expert Output and Weighted Aggregation . . . 7
4.5 Verification and Feedback . . . 8

5 Prototype Implementation: DENA v0 **8**
5.1 Prototype Experts . . . 9
5.2 Prototype Router, Aggregator, Verifier, and Memory . . . 9

6 DENA-Bench v0 **9**
6.1 Benchmark Categories . . . 9
6.2 Compared Systems . . . 10
6.3 Metrics . . . 10

7 Experimental Results **10**
7.1 Interpretation . . . 10
7.2 Reading the Benchmark Figures . . . 11
7.3 Claims Supported by the Results . . . 11

7.4 Claims Not Supported by the Results 11

8 Comparison with Existing Methods **12**

8.1 Summary of Differences 12

9 Novelty and Research Contribution **12**

10 Limitations and Scope **13**

11 Safety and Ethical Considerations **13**

12 Future Work **14**

13 Conclusion **14**

A DENA v0 Pseudocode **15**

B Suggested Citation **15**

1 Introduction

Large-scale artificial intelligence has historically been dominated by dense models in which most parameters participate in most computations. Dense transformer-based models have achieved strong general-purpose capabilities, but they also create practical challenges: updating domain-specific knowledge may require fine-tuning large models, interpretability is limited, and adapting the system to specialized tasks often requires external tools, retrieval systems, or additional model pipelines.

At the same time, modern AI practice has moved toward increasingly modular systems. Retrieval-augmented generation connects models to external knowledge stores. Tool-using agents call calculators, search engines, APIs, and code interpreters. Model orchestration systems route subtasks to specialized models. Mixture-of-Experts architectures activate subsets of expert components. Human feedback pipelines modify model behavior through preference signals. In practice, many deployed AI systems already combine several of these ideas.

However, these components are often integrated informally. A system may have a router, a set of tools, an evaluator, a memory store, and a feedback loop, but the relationships among these components are rarely formalized as a unified architecture. This makes such systems difficult to compare, benchmark, reproduce, and extend.

This paper introduces DENA as a system-level architecture for distributed expert AI systems. DENA treats expert modules as independently trainable, replaceable, and deployable subsystems. A central orchestration mechanism decomposes tasks, a router selects relevant experts, an aggregator combines expert outputs, a verifier evaluates reliability, and a memory-feedback loop records outcomes for future improvement.

1.1 Scope of the Claim

DENA should not be interpreted as a new low-level neural architecture in the same sense as the Transformer, convolutional networks, or recurrent networks. It does not introduce a new computational primitive. Instead, DENA is a **system-level architecture and research framework** that organizes existing primitives into a structured, measurable, and modular design pattern.

This distinction is important. The value of DENA is not that it replaces MoE, RAG, tool agents, or model orchestration systems. Rather, DENA provides a common structure for combining these approaches and evaluating them under shared concepts such as routing quality, expert coverage, cost-aware activation, output aggregation, verification, and feedback-driven modular updating.

1.2 Contributions

This paper makes four contributions:

1. **Architecture:** It proposes DENA as a system-level architecture for distributed expert AI systems.
2. **Formalization:** It defines mathematical components for task decomposition, expert routing, weighted aggregation, verification, memory, and feedback.
3. **Prototype:** It implements DENA v0 as a minimal working prototype using modular expert components.

4. **Benchmark:** It introduces DENA-Bench v0, a controlled benchmark for evaluating routing quality, output quality, estimated cost, and latency.

The current work is preliminary. Its goal is to establish a clear research framework and baseline implementation rather than to claim state-of-the-art performance.

2 Background and Related Work

DENA is related to several existing research directions. This section positions DENA relative to dense models, Mixture-of-Experts systems, retrieval-augmented generation, tool-using agents, model orchestration, multi-agent systems, federated learning, knowledge distillation, and human feedback learning.

2.1 Dense Transformer Models

The Transformer architecture introduced self-attention as a powerful mechanism for sequence modeling and enabled large-scale parallel training [1]. Dense transformer models use a single parameter space to process broad classes of tasks. These systems are powerful but are not inherently modular at the system level. Domain-specific updates, tool use, verification, and memory are usually added through external mechanisms.

DENA differs by making modularity explicit. Instead of assuming that one dense model should handle all computation, DENA allows multiple expert subsystems to be independently trained, updated, and selected.

2.2 Mixture of Experts

Mixture-of-Experts (MoE) systems use conditional computation to select a subset of internal expert networks. Sparse MoE architectures improve scaling by activating only a small number of experts per token or input [2]. Switch Transformer further simplified routing by routing tokens to a single expert in large-scale transformer models [3].

DENA is inspired by expert selection but generalizes it to a system-level setting. In many MoE systems, experts are internal subnetworks within one model and are trained as part of a shared architecture. In DENA, experts may be independent models, tools, retrieval systems, classifiers, verifiers, symbolic engines, or human-supervised modules. They may differ in architecture, training data, modality, cost, latency, and update schedule.

2.3 Retrieval-Augmented Generation

Retrieval-Augmented Generation (RAG) combines parametric language models with external knowledge retrieval [4]. It improves factual grounding by retrieving relevant documents before generation. RAG is especially useful when knowledge changes frequently or when responses require external evidence.

DENA can include RAG systems as expert modules. For example, a legal expert may use retrieval over legal documents, while a research expert may retrieve academic papers. DENA is broader than RAG because it routes not only to documents but also to computational expert subsystems.

2.4 Tool-Using Agents

Tool-using agents allow language models to call calculators, APIs, search engines, code interpreters, and other external tools. ReAct combines reasoning and acting [5], while Toolformer trains models to decide when and how to use APIs [6].

DENA overlaps with tool-using agents but frames tools as one category of expert. A tool may be selected by the router, executed as an expert, and evaluated by the verifier. DENA adds a structured aggregation and feedback loop around such tool use.

2.5 Model Orchestration Systems

HuggingGPT and similar systems use a language model as a controller to decompose tasks, select models, execute subtasks, and summarize outputs [7]. This is closely related to DENA. The key difference is that DENA is presented as a formal system-level architecture with explicit expert metadata, routing functions, aggregation mechanisms, verification, memory, and benchmark metrics.

DENA should therefore be seen not as the first attempt at model orchestration, but as a formalization of model orchestration into a reusable architecture and evaluation framework.

2.6 Multi-Agent Systems

Multi-agent AI systems coordinate multiple agents that communicate, debate, collaborate, or divide tasks. Such systems can be flexible but may be difficult to evaluate and control. AutoGen is one example of a framework for multi-agent conversation and orchestration [8]. DENA differs by using structured expert calls mediated through routing and aggregation rather than relying primarily on open-ended agent conversation.

2.7 Federated and Distributed Learning

Federated learning trains models across decentralized data sources while preserving local data privacy [9]. DENA is distributed in a different sense: it distributes expertise across modules rather than distributing training data across clients. However, DENA could be combined with federated learning if experts are trained locally and coordinated by a central router.

2.8 Knowledge Distillation and Human Feedback

Knowledge distillation transfers knowledge from larger models or ensembles into smaller models [10]. DENA could act as a teacher system whose aggregated outputs are distilled into a smaller student model. Conversely, distilled experts could be used inside DENA for efficient deployment.

Reinforcement Learning from Human Feedback (RLHF) and preference optimization methods use human judgments to improve model behavior [11]. DENA incorporates feedback at the system

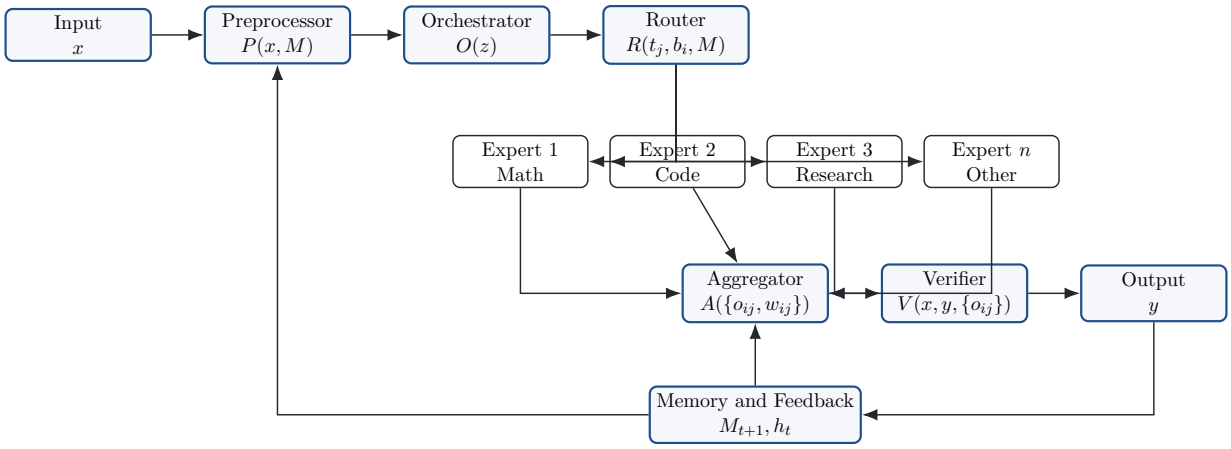

Figure 1: System-level view of the DENA pipeline. The diagram separates task understanding, expert selection, expert execution, aggregation, verification, and feedback storage into explicit modules.

Figure interpretation. The top row shows the main inference path from input to router. The middle row shows that experts are independent subsystems rather than hidden layers inside one monolithic model. The bottom row shows that outputs and feedback are stored in memory and can influence later routing or aggregation. This figure should be read as a system architecture: each block may be implemented with rules, neural models, retrieval systems, tools, or human-in-the-loop components.

level. Feedback may update expert quality scores, improve routing policies, calibrate confidence, or retrain specific expert modules.

3 DENA Architecture

DENA consists of a set of expert subsystems coordinated by orchestration, routing, aggregation, verification, memory, and feedback modules. Figure 1 shows the high-level structure.

3.1 High-Level Flow

The DENA inference pipeline is:

Input → Preprocessor → Orchestrator → Router → Selected Experts → Aggregator → Verifier → Output → Memory/Feedback.

This pipeline is intended to be general. Individual implementations may use neural routers, rule-based routers, embedding-based retrieval, external tools, human review, or learned aggregation.

3.2 How to Interpret the Architecture Diagram

Figure 1 is not intended to describe a single fixed neural network layer. Instead, it defines the information flow of a distributed expert system. The router is responsible for selecting a subset of experts, the experts produce domain-specific outputs, the aggregator combines these outputs, and the verifier decides whether the result is acceptable. The feedback path is shown explicitly

because DENA treats learning and system improvement as modular: the router, expert metadata, aggregator, verifier, or individual experts may be updated without retraining the full system.

3.3 Core Components

Preprocessor. The preprocessor transforms raw input into a task representation. This may include tokenization, embedding, classification, document parsing, modality conversion, or memory retrieval.

Orchestrator. The orchestrator decomposes a user query or input task into one or more subtasks. For example, a query asking for legal and financial analysis may be decomposed into legal interpretation, financial risk analysis, and final summary generation.

Router. The router selects relevant experts for each subtask. Selection may be based on expert domain, task similarity, confidence, cost, latency, historical performance, availability, or safety constraints.

Expert subsystems. An expert subsystem is any specialized computational module that can process a task and return an output. Experts may include fine-tuned language models, dense models, domain-specific models, retrieval systems, symbolic solvers, calculators, code interpreters, classifiers, databases, rule engines, or human-reviewed modules.

Aggregator. The aggregator combines expert outputs into a final answer or intermediate result. Aggregation may use confidence weighting, voting, synthesis, ranking, verifier-guided selection, or a learned aggregation model.

Verifier. The verifier checks the reliability, consistency, safety, or correctness of outputs. If the verifier rejects an output, DENA may reroute the task, call additional experts, request human review, or return uncertainty.

Memory and feedback. The memory system stores previous interactions, routing decisions, expert outputs, feedback, benchmark results, and performance traces. Feedback may come from users, humans, evaluators, automatic scoring functions, or benchmark results.

4 Mathematical Formulation

This section defines a general mathematical structure for DENA.

4.1 System Definition

Let a DENA system be defined as:

$$\mathcal{D} = (\mathcal{X}, \mathcal{Y}, \mathcal{E}, P, O, R, A, V, M, F, U), \tag{1}$$

where $\mathcal{X}$ is the input space, $\mathcal{Y}$ is the output space, $\mathcal{E} = \{E_1, E_2, \ldots, E_n\}$ is the set of experts, P is the preprocessing function, O is the orchestrator, R is the router, A is the aggregator, V is the verifier, M is the memory system, F is the feedback function, and U is the update mechanism.

Each expert E_i has metadata:

$$b_i = (d_i, q_i, c_i, \ell_i, a_i), \tag{2}$$

where d_i is the expert domain representation, q_i is an estimated quality score, c_i is the computational cost, ℓ_i is the latency estimate, and a_i is the availability indicator.

4.2 Preprocessing and Task Decomposition

Given an input x, the preprocessor produces a representation:

$$z = P(x, M_t). \tag{3}$$

The orchestrator decomposes z into subtasks:

$$T = O(z) = \{\tau_1, \tau_2, \ldots, \tau_m\}. \tag{4}$$

4.3 Expert Routing

For each subtask τ_j, the router scores each expert E_i:

$$s_{ij} = g_\psi(\tau_j, b_i, M_t), \tag{5}$$

where g_ψ is a routing score function. The routing probability is:

$$p_{ij} = \frac{\exp(s_{ij})}{\sum_{k=1}^{n} \exp(s_{kj})}. \tag{6}$$

The selected expert set for subtask τ_j is:

$$S_j = \mathrm{TopK}(\{p_{1j}, p_{2j}, \ldots, p_{nj}\}, K). \tag{7}$$

A cost-aware routing score can be written as:

$$\hat{s}_{ij} = s_{ij} + \alpha q_i - \beta c_i - \gamma \ell_i, \tag{8}$$

where α, β, and γ control quality reward, cost penalty, and latency penalty.

4.4 Expert Output and Weighted Aggregation

Each selected expert produces an output and confidence:

$$(o_{ij}, u_{ij}) = E_i(\tau_j), \quad i \in S_j. \tag{9}$$

An aggregation weight may be defined as:

$$w_{ij} = \text{softmax}(\eta_1 p_{ij} + \eta_2 u_{ij} + \eta_3 q_i - \eta_4 c_i - \eta_5 \ell_i). \tag{10}$$

The aggregated result for a subtask is:

$$r_j = A(\{(o_{ij}, w_{ij}) : i \in S_j\}). \tag{11}$$

The final output is:

$$y = A_{\text{global}}(\{r_1, r_2, \ldots, r_m\}, z, M_t). \tag{12}$$

4.5 Verification and Feedback

The verifier assigns a score:

$$v = V(x, y, \{o_{ij}\}). \tag{13}$$

The output is accepted if:

$$v \geq \rho, \tag{14}$$

where ρ is an acceptance threshold. If verification fails, DENA may reroute, retrieve more context, request another expert, or ask for human review.

A feedback signal is represented as:

$$h_t = F(x_t, y_t). \tag{15}$$

The update mechanism modifies some subset of the system:

$$U(h_t) \to \Delta R, \Delta A, \Delta V, \Delta M, \text{or } \Delta E_i. \tag{16}$$

A key property of DENA is modular updating. If only expert E_k needs improvement, DENA may update E_k while leaving other experts unchanged:

$$\Delta\theta_i = 0 \quad \forall i \neq k. \tag{17}$$

5 Prototype Implementation: DENA v0

A minimal DENA prototype was implemented to test the architecture. The prototype is not intended to be a production AI system. It is a proof-of-architecture implementation.

5.1 Prototype Experts

DENA v0 includes five simplified experts:

- `Expert_Math`
- `Expert_Code`
- `Expert_Definition`
- `Expert_Research`
- `Expert_General`

Each expert has metadata including name, domain, keywords, quality score, cost, latency, and availability.

5.2 Prototype Router, Aggregator, Verifier, and Memory

The router uses a heuristic scoring function based on domain match, keyword relevance, expert quality score, estimated cost, estimated latency, and memory bonus. The router selects the top-K experts for each subtask.

The aggregator uses weighted synthesis based on routing probability, expert confidence, expert quality, cost, and latency. The verifier computes a reliability score and accepts or rejects the output based on a threshold. A JSONL-based memory store records input queries, subtasks, selected experts, outputs, verification scores, and acceptance status.

6 DENA-Bench v0

DENA-Bench v0 is a controlled benchmark designed to evaluate DENA's routing, answer quality, and estimated efficiency.

6.1 Benchmark Categories

The benchmark contains tasks from five categories:

1. `math_exact`
2. `code_debug`
3. `definition`
4. `architecture_research`
5. `mixed_domain`

6.2 Compared Systems

The benchmark compares `DENA_top1`, `DENA_top2`, `DENA_top3`, `AllExperts_proxy`, `GeneralOnly_proxy`, `RandomRouter_proxy`, and `OracleRouter_upper_bound`. The oracle router is not a deployable system; it uses expected expert labels as an upper-bound reference.

6.3 Metrics

Table 1 defines the main benchmark metrics.

Table 1: Primary metrics used in DENA-Bench v0. The benchmark evaluates both output quality and routing behavior, because a distributed expert system should be judged not only by its final answer but also by whether it selected appropriate experts efficiently.

Metric	Definition
Routing precision	$\lvert\hat{S} \cap S^*\rvert / \lvert\hat{S}\rvert$
Routing recall	$\lvert\hat{S} \cap S^*\rvert / \lvert S^*\rvert$
Routing F1	Harmonic mean of precision and recall
Answer score	Match between output and expected keywords or exact answers
Quality score	$0.45\cdot$ RoutingRecall $+0.45\cdot$ AnswerScore $+0.10\cdot$ Accepted
Cost-adjusted score	Quality $-0.10\cdot$ Cost $-0.05\cdot$ Latency

7 Experimental Results

The preliminary benchmark produced the results shown in Table 2. These results should be interpreted as prototype validation rather than evidence of general superiority.

Table 2: DENA-Bench v0 preliminary results. Higher values are better for quality, cost-adjusted score, routing F1, and answer score; lower values are better for average cost and latency. These values are prototype-level estimates and should not be interpreted as production hardware measurements.

System	Quality	Cost-Adjusted	Routing F1	Answer	Avg Cost	Avg Latency
DENA_top1	0.961	0.932	0.903	0.939	0.194	0.194
DENA_top2	0.974	0.928	0.741	0.949	0.337	0.243
DENA_top3	0.974	0.911	0.602	0.949	0.485	0.300
OracleRouter_upper_bound	0.950	0.908	0.733	0.929	0.309	0.227
AllExperts_proxy	0.977	0.883	0.420	0.949	0.788	0.315
RandomRouter_proxy	0.741	0.701	0.432	0.803	0.299	0.213
GeneralOnly_proxy	0.302	0.293	0.000	0.449	0.063	0.063

7.1 Interpretation

`DENA_top2` achieved quality close to `AllExperts_proxy` while using substantially lower estimated cost. This supports the hypothesis that sparse routing can approximate all-expert execution in a controlled setting.

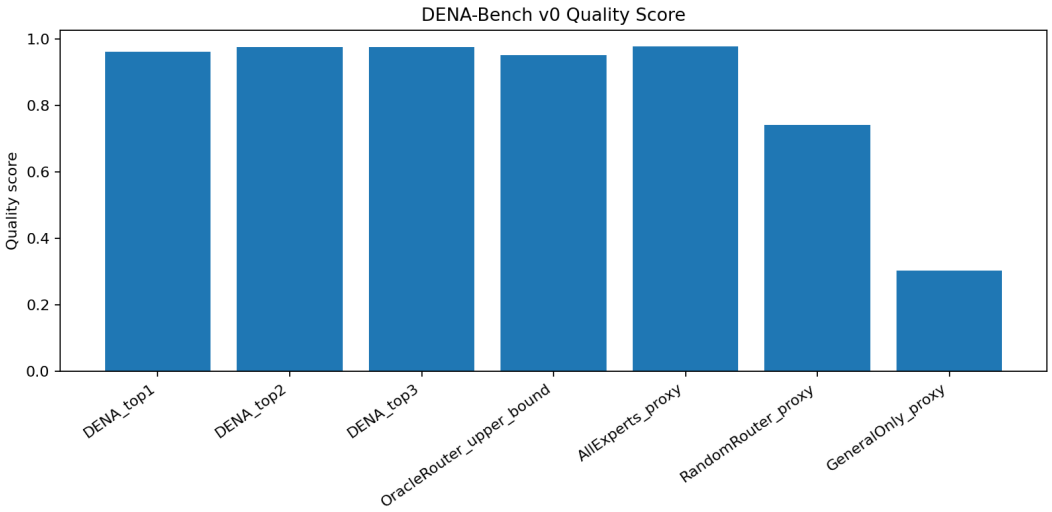

Figure 2: Quality score comparison across benchmarked systems. The quality score combines routing recall, answer score, and verifier acceptance into a single prototype-level measure.

> **Figure interpretation.** Higher is better. The all-expert proxy obtains the highest raw quality because every expert is always activated. However, `DENA_top1`, `DENA_top2`, and `DENA_top3` remain close while activating fewer experts. This supports the preliminary claim that sparse expert routing can preserve much of the benefit of all-expert execution in the controlled benchmark.

`DENA_top1` achieved the highest cost-adjusted score, suggesting that sparse expert selection may offer strong efficiency when routing is accurate. `AllExperts_proxy` achieved high quality but low routing F1 because it activates unnecessary experts. `GeneralOnly_proxy` performed weakest because it lacks access to specialized experts.

7.2 Reading the Benchmark Figures

Figures 2, 3, and 4 should be interpreted together. Figure 2 shows raw prototype-level quality, Figure 3 shows the estimated cost of activating experts, and Figure 4 shows whether the selected experts were appropriate. A system that scores well on quality but poorly on routing or cost may still be inefficient. DENA-Bench therefore evaluates the tradeoff between answer quality and expert-selection efficiency.

7.3 Claims Supported by the Results

The results show that DENA v0 is implementable, measurable, interpretable, and capable of cost-aware routing in a controlled benchmark.

7.4 Claims Not Supported by the Results

The results do not prove that DENA outperforms production dense models, large-scale MoE systems, or mature agent frameworks. The experiments use simplified expert modules and synthetic benchmark tasks.

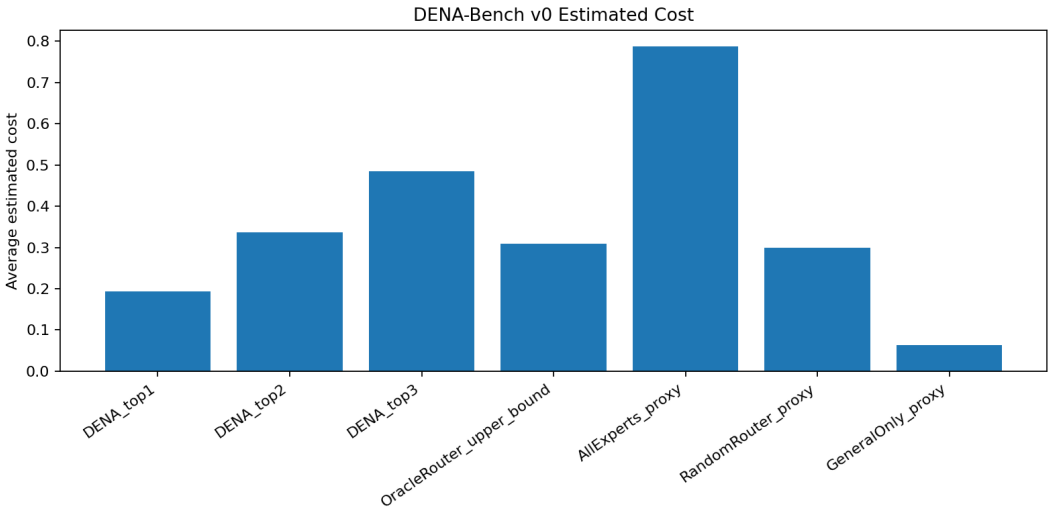

Figure 3: Estimated expert activation cost across benchmarked systems. Cost is a synthetic proxy derived from expert metadata in the prototype, not a measured dollar cost or hardware FLOP count.

Figure interpretation. Lower is better. `AllExperts_proxy` has the highest estimated cost because it activates every expert for every task. The DENA variants activate fewer experts and therefore reduce estimated activation cost. This plot explains why cost-adjusted scoring is useful: raw quality alone can hide inefficient expert selection.

8 Comparison with Existing Methods

DENA is best understood as a system-level architecture that unifies patterns already present in modern AI pipelines. Table 3 summarizes its relationship to major existing approaches.

8.1 Summary of Differences

DENA does not claim to invent orchestration, routing, expert models, verification, or feedback individually. Its contribution is the **formal unification** of these components into a system-level architecture with benchmarkable behavior.

9 Novelty and Research Contribution

The novelty of DENA lies in four areas.

Unified formalization. DENA gives a unified structure to expert routing, aggregation, verification, memory, and feedback.

Explicit expert abstraction. Experts are treated as independently trainable and replaceable subsystems with metadata such as quality, cost, latency, confidence, and availability.

Benchmarkable routing and cost. DENA-Bench v0 introduces metrics for routing precision, routing recall, answer score, quality score, and cost-adjusted score.

Modular update path. DENA supports targeted improvement of individual experts, routers,

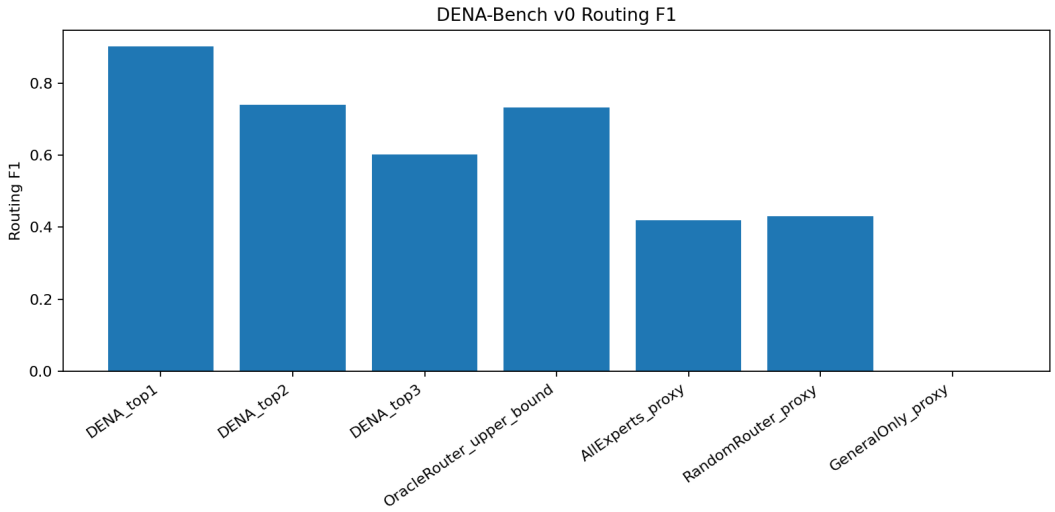

Figure 4: Routing F1 comparison across benchmarked systems. Routing F1 balances expert-selection precision and expert-selection recall.

Figure interpretation. Higher is better. `DENA_top1` achieves the strongest routing F1 because it tends to select fewer unnecessary experts. `AllExperts_proxy` has low routing F1 despite strong output quality because selecting all experts lowers precision. This highlights a central design goal of DENA: a good distributed expert system should identify the right experts, not simply call every available expert.

aggregators, or verifiers without requiring complete retraining of the system.

10 Limitations and Scope

DENA v0 has important limitations.

First, the current expert modules are simplified and rule-based. They are not large-scale neural experts. Second, the router and aggregator are heuristic rather than learned. Third, DENA-Bench v0 uses controlled synthetic tasks rather than large public benchmarks. Fourth, cost and latency are estimated rather than measured on production hardware. Fifth, DENA does not introduce a new neural computation primitive. It is a system-level architecture and framework.

These limitations define the next stage of the research rather than invalidating the architecture.

11 Safety and Ethical Considerations

Distributed expert AI systems introduce safety concerns. Poor routing may select an inappropriate expert. Expert outputs may conflict. A verifier may incorrectly accept unreliable outputs. Feedback signals may be noisy or biased. If used in high-stakes domains such as medicine, law, finance, cybersecurity, or defense, DENA systems should include human oversight, audit logs, uncertainty reporting, access control, and domain-specific validation.

DENA's modularity may improve safety by allowing individual experts to be removed, replaced, or restricted. However, it also increases the number of system interfaces and possible failure points. Future work should study security, robustness, expert trust calibration, and failure isolation.

Table 3: Conceptual comparison between DENA and related methods. The table emphasizes that DENA is not a replacement for these methods; it is a system-level framework that can organize and evaluate them as expert subsystems.

Method	Core Idea	Strength	DENA Difference
Dense models	One model handles most computation	Simple inference and broad capability	DENA separates capabilities into independent experts
MoE	Router selects internal expert subnetworks	Efficient sparse computation	DENA uses distributed, independently replaceable experts
Ensembles	Multiple models are combined	Strong accuracy	DENA selects only relevant experts
RAG	Retrieve documents for generation	Factual grounding	DENA routes to computational experts, including RAG experts
Tool agents	Language model calls tools	Flexible external actions	DENA adds structured routing, aggregation, verification, and feedback
Model orchestration	Controller selects models for subtasks	Modular task execution	DENA formalizes expert metadata and benchmarkable routing
Multi-agent systems	Multiple agents communicate	Flexible collaboration	DENA uses controlled expert calls and aggregation

12 Future Work

Future work will focus on replacing toy experts with neural experts, training the router with supervised and reinforcement learning methods, improving aggregation with learned synthesis models, developing stronger verifiers, expanding DENA-Bench to public datasets, measuring real latency, FLOPs, memory, and cost, comparing against real dense, MoE, RAG, ensemble, and agent baselines, testing modular expert updates, studying expert failure and recovery, and exploring DENA-to-student distillation.

A stronger future version of DENA should demonstrate measurable advantages in modular update efficiency, cost-quality tradeoffs, failure isolation, interpretability, or feedback-driven improvement.

13 Conclusion

This paper proposed DENA, a system-level architecture for distributed expert AI systems. DENA formalizes a modular pipeline consisting of task decomposition, expert routing, expert execution, aggregation, verification, memory, and feedback-driven updates. A prototype implementation and controlled benchmark demonstrate that the architecture is implementable and measurable.

DENA is not a new neural primitive. Its value lies in structuring and formalizing a pattern that is increasingly common in modern AI practice. By providing explicit components, mathematical definitions, prototype code, and benchmark metrics, DENA offers a foundation for systematic research into modular distributed expert AI systems.

Acknowledgments

The author thanks early reviewers and community members whose feedback helped clarify the distinction between neural architectures and system-level architectures.

A DENA v0 Pseudocode

Algorithm: DENA Inference

Input: x
Memory: M
Experts: $\mathcal{E} = \{E_1, E_2, \ldots, E_n\}$

1. $z = \text{Preprocess}(x, M)$
2. $T = \text{Orchestrate}(z)$
3. For each subtask $\tau \in T$:
   (a) Score each expert E_i
   (b) Select top-K experts S
   (c) Execute selected experts
   (d) Aggregate expert outputs
4. Combine subtask outputs into final answer y
5. Verify y
6. Store trace in memory
7. Return y

B Suggested Citation


```
@misc{eada2026dena,
  title={DENA: A System-Level Architecture for Distributed Expert AI Systems},
  author={Eada, Harsha Sai Reddy},
  year={2026},
  doi={10.5281/zenodo.19841838},
  url={https://doi.org/10.5281/zenodo.19841838},
  note={Preprint and software release}
}
```

References

[1] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Lukasz Kaiser, and Illia Polosukhin. Attention Is All You Need. In *Advances in Neural Information Processing Systems*, 2017.

[2] Noam Shazeer, Azalia Mirhoseini, Krzysztof Maziarz, Andy Davis, Quoc Le, Geoffrey Hinton, and Jeff Dean. Outrageously Large Neural Networks: The Sparsely-Gated Mixture-of-Experts Layer. In *International Conference on Learning Representations*, 2017.

[3] William Fedus, Barret Zoph, and Noam Shazeer. Switch Transformers: Scaling to Trillion Parameter Models with Simple and Efficient Sparsity. *Journal of Machine Learning Research*, 2022.

[4] Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Kuttler, Mike Lewis, Wen-tau Yih, Tim Rocktaschel, Sebastian Riedel, and Douwe Kiela. Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks. In *Advances in Neural Information Processing Systems*, 2020.

[5] Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. ReAct: Synergizing Reasoning and Acting in Language Models. In *International Conference on Learning Representations*, 2023.

[6] Timo Schick, Jane Dwivedi-Yu, Roberto Dessi, Roberta Raileanu, Maria Lomeli, Eric Hambro, Luke Zettlemoyer, Nicola Cancedda, and Thomas Scialom. Toolformer: Language Models Can Teach Themselves to Use Tools. In *Advances in Neural Information Processing Systems*, 2023.

[7] Yongliang Shen, Kaitao Song, Xu Tan, Dongsheng Li, Weiming Lu, and Yueting Zhuang. HuggingGPT: Solving AI Tasks with ChatGPT and its Friends in Hugging Face. In *Advances in Neural Information Processing Systems*, 2023.

[8] Qingyun Wu, Gagan Bansal, Jieyu Zhang, Yiran Wu, Beibin Li, Erkang Zhu, Li Jiang, Xiaoyun Zhang, Shaokun Zhang, Jiale Liu, Ahmed H. Awadallah, Ryen W. White, Doug Burger, and Chi Wang. AutoGen: Enabling Next-Gen LLM Applications via Multi-Agent Conversation. arXiv preprint arXiv:2308.08155, 2023.

[9] Brendan McMahan, Eider Moore, Daniel Ramage, Seth Hampson, and Blaise Aguera y Arcas. Communication-Efficient Learning of Deep Networks from Decentralized Data. In *Artificial Intelligence and Statistics*, 2017.

[10] Geoffrey Hinton, Oriol Vinyals, and Jeff Dean. Distilling the Knowledge in a Neural Network. arXiv preprint arXiv:1503.02531, 2015.

[11] Long Ouyang, Jeffrey Wu, Xu Jiang, Diogo Almeida, Carroll Wainwright, Pamela Mishkin, Chong Zhang, Sandhini Agarwal, Katarina Slama, Alex Ray, John Schulman, Jacob Hilton, Fraser Kelton, Luke Miller, Maddie Simens, Amanda Askell, Peter Welinder, Paul Christiano, Jan Leike, and Ryan Lowe. Training Language Models to Follow Instructions with Human Feedback. In *Advances in Neural Information Processing Systems*, 2022.

[12] Tom B. Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared Kaplan, Prafulla Dhariwal, Arvind Neelakantan, Pranav Shyam, Girish Sastry, Amanda Askell, Sandhini Agarwal, Ariel Herbert-Voss, Gretchen Krueger, Tom Henighan, Rewon Child, Aditya Ramesh, Daniel Ziegler,

Jeffrey Wu, Clemens Winter, Chris Hesse, Mark Chen, Eric Sigler, Mateusz Litwin, Scott Gray, Benjamin Chess, Jack Clark, Christopher Berner, Sam McCandlish, Alec Radford, Ilya Sutskever, and Dario Amodei. Language Models are Few-Shot Learners. In *Advances in Neural Information Processing Systems*, 2020.